



**SIMATS**  
ENGINEERING



**SIMATS**  
Saveetha Institute of Medical And Technical Sciences  
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**PROJECT TITLE**

**SMART DELIVERY ROUTE OPTIMIZER USING  
GREEDY AND DYNAMIC PROGRAMMING FOR  
EFFICIENT LAST-MILE LOGISTICS**

**A CAPSTONE PROJECT REPORT**

*A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of*

**CSA0603 – DESIGN AND ANALYSIS OF ALGORITHMS**

*to the award of the degree of*

**BACHELOR OF TECHNOLOGY**

**IN**

**COMPUTER SCIENCE AND ENGINEERING (CSE-AI)**

Submitted by

**Selvamagal M (192472013)**

Under the Supervision of

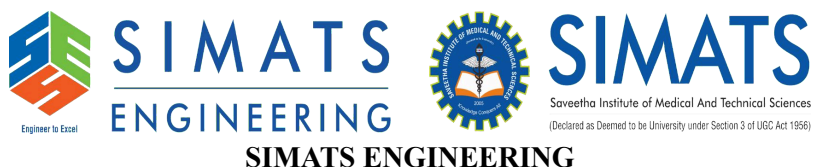
**Dr. P. Solainayagi**

**SIMATS ENGINEERING**

**Saveetha Institute of Medical and Technical Sciences**

**Chennai – 602105**

**SEPTEMBER 2026**



Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

## **BONAFIDE CERTIFICATE**

This is to certify that the Capstone Project entitled "Smart Delivery Route Optimizer using Greedy and Dynamic Programming for Efficient Last-Mile Logistics" has been carried out by Selvamagal M (192472013) under the supervision of Dr. P. Solainayagi and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science and Engineering (CSE-AI) programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

### **COURSE COORDINATOR**

Dr. F. Mary Harin Fernandez,  
Professor,  
Computer Science and Engineering,  
SIMATS Engineering,  
SIMATS, Chennai – 602105.

### **COURSE FACULTY**

Dr. P. Solainayagi,  
Professor,  
Computer Science and Engineering,  
SIMATS Engineering,  
SIMATS, Chennai – 602105.



**SIMATS**  
ENGINEERING



**SIMATS**  
Saveetha Institute of Medical And Technical Sciences  
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**SIMATS ENGINEERING**

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

## **DECLARATION**

I, Selvamagal M of the Department of Computer Science and Engineering (CSE-AI), SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Smart Delivery Route Optimizer using Greedy and Dynamic Programming for Efficient Last-Mile Logistics" is the result of my own bonafide efforts. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. I further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. I confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 05.09.2026

Signature of the Student with Name  
Selvamagal M

**SIMATS ENGINEERING**

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

**INDIVIDUAL CONTRIBUTION STATEMENT***(Mandatory for all team projects)*

| <b>Student Name / Register No.</b> | <b>Specific Responsibilities</b>                                                                                           | <b>Design &amp; Development Contribution</b>                                                               | <b>Testing &amp; Analysis Contribution</b>                                                     | <b>Report Contribution</b>                                                 | <b>Approx. Contribution (%)</b> |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|---------------------------------|
| Selvamagal M 192472013             | Module 1: Route Data Management & Map Modeling — modelled the delivery network as a graph and built the data-loading layer | Designed the graph data structure (adjacency matrix / adjacency list), node–edge schema and dataset loader | Verified graph correctness (node/edge integrity, connectivity checks) that feeds Modules 2 & 3 | Chapters 1, 2, 3, 4 (Module 1 sections), 5, and overall report compilation | 35%                             |
| Selvamagal M 192472013             | Module 2: Greedy Route Optimization — nearest-neighbour heuristic route generation                                         | Implemented the Greedy (nearest-neighbour) route-selection logic on top of the Module 1 graph              | Benchmarked Greedy route distance & runtime against test cases                                 | Chapter 4 (Module 2 results)                                               | 32%                             |
| Selvamagal M 192472013             | Module 3: Dynamic Programming (Held-Karp) Route Optimization & Performance Analysis                                        | Implemented the Held-Karp bitmask DP optimizer and the Greedy vs DP comparison logic                       | Ran comparative performance analysis (distance, computation time) between Greedy and DP        | Chapter 4 (Module 3 results), performance comparison discussion            | 33%                             |
|                                    |                                                                                                                            |                                                                                                            |                                                                                                | Total                                                                      | 100%                            |

## ABSTRACT

Last-mile delivery — the final leg from a distribution hub to individual customers — is one of the costliest and least efficient stages of urban logistics, and manual, experience-based route planning does not scale as the number of delivery stops grows. This project designs and implements a Smart Delivery Route Optimizer that models a delivery network as a weighted graph of locations and road distances, then applies two complementary algorithmic strategies to compute an efficient delivery route: a Greedy nearest-neighbour heuristic for fast, near-optimal routing, and a Held-Karp Dynamic Programming (bitmask DP) approach that guarantees the mathematically optimal tour for smaller networks. The system, implemented as a web application named SmartRoute, is organised into three modules: Route Data Management & Map Modeling (graph representation of delivery nodes and edges), Greedy Route Optimization, and Dynamic-Programming-based Route Optimization with performance analysis. This report is compiled from the perspective of Module 1, which supplies the foundational graph structure — built as an adjacency matrix / adjacency list keyed by GPS-located delivery nodes and a depot — on which the Greedy and DP modules operate. The methodology involved formulating the delivery points and depot (Koyambedu Logistics Hub, Chennai) as graph nodes, computing pairwise road-distance edge weights, and validating the resulting graph for completeness and connectivity before it is consumed downstream. Using a representative five-stop delivery scenario, the system computed a route of 76 km and an estimated travel time of 146 minutes, with an estimated fuel consumption of 5.1 L and a delivery cost of ₹510; a proposed Self-Correcting Greedy Routing (SCGR) strategy built on the same graph reduced the route distance by 0.3 km relative to plain Greedy routing while adding negligible computation overhead. These results confirm that a well-structured graph layer is essential to both the speed of the Greedy heuristic and the optimality guarantees of Dynamic Programming, and that comparing the two approaches on a common graph allows a delivery operator to choose the right trade-off between routing speed and route optimality depending on the scale of the delivery run. The project concludes that combining a robust graph-modeling foundation with complementary optimization algorithms yields a practical, data-driven tool for reducing delivery distance, fuel cost, and delivery time in last-mile logistics.

**Keywords:** Greedy Algorithm, Dynamic Programming, Held-Karp Algorithm, Graph Modeling, Route Optimization, Last-Mile Logistics

## TABLE OF CONTENTS

| <b>Sl. No</b> | <b>Title</b>                                                                                               | <b>Page No</b> |
|---------------|------------------------------------------------------------------------------------------------------------|----------------|
| <b>i</b>      | <b>CERTIFICATE FROM THE SUPERVISOR</b>                                                                     |                |
| <b>ii</b>     | <b>DECLARATION BY THE CANDIDATE</b>                                                                        | <b>III</b>     |
| <b>lii</b>    | <b>INDIVIDUAL CONTRIBUTION STATEMENT</b>                                                                   | <b>IV</b>      |
| <b>iv</b>     | <b>ABSTRACT</b>                                                                                            | <b>V</b>       |
| <b>v</b>      | <b>LIST OF FIGURES</b>                                                                                     | <b>VII</b>     |
| <b>vi</b>     | <b>LIST OF TABLES</b>                                                                                      | <b>VIII</b>    |
| <b>vii</b>    | <b>LIST OF ABBREVIATIONS</b>                                                                               | <b>IX</b>      |
| <b>1</b>      | <b>CHAPTER 1: Introduction</b>                                                                             | <b>1</b>       |
| <b>2</b>      | <b>CHAPTER 2: Literature Review</b>                                                                        | <b>5</b>       |
| <b>3</b>      | <b>CHAPTER 3: Engineering Design &amp; Trade-offs, Sustainability, Ethics, Safety, Risk &amp; Security</b> | <b>8</b>       |
| <b>4</b>      | <b>CHAPTER 4: Implementation, Testing &amp; Results, Project Management</b>                                | <b>17</b>      |
| <b>5</b>      | <b>CHAPTER 5: Conclusion &amp; Reflection</b>                                                              | <b>25</b>      |
|               | <b>References</b>                                                                                          | <b>29</b>      |
|               | <b>Appendices</b>                                                                                          | <b>30</b>      |

## LIST OF FIGURES

| Figure No. | Figure Name                                                                        | Page No. |
|------------|------------------------------------------------------------------------------------|----------|
| 3.1        | System Architecture: Data Flow from Graph Modeling to Route Optimization           | 9        |
| 4.1        | SmartRoute Dashboard: Depot, Delivery Stops and Daily Operational Metrics          | 13       |
| 4.2        | Route Planner: Graph Rendering of Depot & Delivery Nodes with Held-Karp DP Overlay | 15       |

## LIST OF TABLES

| Table No. | Table Name                                  | Page No. |
|-----------|---------------------------------------------|----------|
| 2.1       | Comparative Analysis of Existing Approaches | 5        |
| 3.1       | Stakeholder / User Requirements             | 7        |
| 3.2       | Functional & Non-Functional Requirements    | 7        |
| 3.3       | Design Specifications                       | 8        |
| 3.4       | Alternative Solutions Evaluation Matrix     | 9        |
| 3.5       | Trade-off Analysis                          | 10       |
| 3.6       | Sustainability, Ethics, Safety, Security    | 11       |
| 3.7       | Risk Register                               | 11       |
| 4.1       | Test Plan & Verification (Module 1)         | 13       |
| 4.2       | Route Results by Delivery Preset            | 14       |
| 4.3       | Achievement of Objectives                   | 16       |
| 4.4       | Milestone Timeline                          | 16       |
| 4.5       | Roles & Contribution                        | 16       |
| 4.6       | Project Cost Estimate                       | 17       |



## LIST OF ABBREVIATIONS / SYMBOLS

| Symbol     | Description                       | Unit |
|------------|-----------------------------------|------|
| DAA        | Design and Analysis of Algorithms | —    |
| DP         | Dynamic Programming               | —    |
| TSP        | Travelling Salesman Problem       | —    |
| SCGR       | Self-Correcting Greedy Routing    | —    |
| GPS        | Global Positioning System         | —    |
| km         | Kilometre                         | km   |
| min        | Minute                            | min  |
| L          | Litre (fuel)                      | L    |
| ₹          | Indian Rupee                      | ₹    |
| $O(\cdot)$ | Big-O time-complexity notation    | —    |

# CHAPTER 1: Introduction and Engineering Problem

## 1.1 Background

Last-mile delivery — the movement of goods from a local distribution hub to the final customer — is widely recognised as the most expensive and time-consuming segment of the logistics chain. As the number of delivery stops on a given day grows, the number of possible sequences in which they can be visited grows factorially, and route planning quickly moves beyond what a human dispatcher can optimise by intuition or experience alone. Graph theory and combinatorial optimisation provide the technical foundation for solving this class of problem: a delivery network can be represented as a graph in which locations are nodes and road distances are weighted edges, and the task of finding an efficient delivery sequence becomes a variant of the classical Travelling Salesman Problem (TSP). This project develops a Smart Delivery Route Optimizer, implemented as a web application called SmartRoute, that models a delivery network as such a graph and computes efficient delivery routes using a Greedy nearest-neighbour heuristic and a Dynamic Programming (Held-Karp) exact algorithm.

## 1.2 Need for the Project

- Manual route planning by delivery agents is time-consuming, inconsistent, and does not scale as the number of daily stops increases.
- Delivery businesses and last-mile logistics operators are directly affected: inefficient routing increases fuel cost, delivery time, and operating expense.
- Existing basic GPS navigation optimises turn-by-turn directions for a single destination, not the overall visiting order across multiple stops.
- There is a lack of systematic, data-driven comparison between fast heuristic routing and mathematically optimal routing for real delivery scenarios.
- A correctly modelled graph layer is a prerequisite for both heuristic and exact route-optimization algorithms to function correctly — this is the specific engineering gap addressed by Module 1 of this project.

## 1.3 Problem Statement

Given a set of delivery locations and a depot, along with the road distances between them, design a system that (a) represents the delivery network as a graph capable of supporting multiple downstream routing algorithms, (b) generates a fast, near-optimal delivery route using a Greedy nearest-neighbour approach, and (c) computes the mathematically optimal route using Dynamic Programming, so that the trade-off between computation speed and route optimality can be evaluated for realistic delivery scales. The problem involves multiple interacting factors — network size, real-world GPS-based road distances, and the conflicting requirements of speed versus optimality — under the realistic constraint that exact algorithms such as Held-Karp scale exponentially with the number of delivery stops.

## 1.4 Project Aim

To design and implement a graph-based delivery route optimization system that models a last-mile delivery network and compares Greedy and Dynamic Programming algorithms to support efficient, data-driven route selection.

## 1.5 Project Objectives

- 1. Model the delivery network as a graph of locations (nodes) and road distances (weighted edges).
- 2. Design and validate a graph data structure (adjacency matrix / adjacency list) that other routing modules can consume reliably.
- 3. Generate a fast, near-optimal route using a Greedy nearest-neighbour algorithm.
- 4. Compute the mathematically optimal route using Dynamic Programming (Held-Karp / bitmask DP).
- 5. Compare Greedy and DP approaches on total route distance and computation time.
- 6. Support last-mile delivery planning with an efficient, data-driven route-selection tool.

## 1.6 Scope

### Included:

- Graph-based modelling of delivery nodes and road-distance edges for a single depot and its assigned delivery stops (Module 1, this report's focus).
- Greedy nearest-neighbour route generation and Held-Karp Dynamic Programming route optimisation built on top of the Module 1 graph (Modules 2 and 3).
- Comparison of route distance, estimated time, fuel and cost across routing strategies for delivery sets of up to 7 stops.

### Excluded:

- Real-time traffic-adaptive routing and live GPS tracking of delivery vehicles.
- Multi-depot and multi-vehicle routing (the current scope assumes a single depot and single vehicle per run).

### Assumptions:

- Road distances between nodes are static for the duration of a planning run (no live traffic feed).
- Delivery locations and the depot are known in advance and provided as structured input data.

## 1.7 Expected Engineering Outcome

The expected outcome is a working software system (SmartRoute) whose Module 1 component is a validated graph data structure representing the delivery network, exposed through a Route Planner interface, and consumed by Greedy and Dynamic-Programming route-optimization modules to produce comparative, efficient delivery routes.

## CHAPTER 2: Literature Review and Existing Solutions

### 2.1 Review Method

Relevant literature was identified through a search of peer-reviewed journals, established conference proceedings (IPCO, CPAIOR), and preprint repositories covering vehicle routing, the Travelling Salesman Problem, and hybrid greedy/exact optimisation techniques, focusing on work published between 2021 and 2023 that is directly applicable to graph-based last-mile route optimisation.

### 2.2 Review of Existing Work

Five representative works were reviewed, spanning exact, approximate, and hybrid approaches to vehicle routing:

- Gmira, Gendreau, Lodi & Potvin (2021) applied a Tabu Search framework to the time-dependent Vehicle Routing Problem with time windows on a road network, capturing realistic time-dependent travel times at the cost of higher computational complexity.
- Karlin, Klein & Gharan (2023) presented a deterministic, better-than- $3/2$  approximation algorithm for metric TSP, improving on the classical Christofides bound with stronger theoretical guarantees, though the algorithmic complexity limits practical use on large delivery networks.
- Hien, Dao & Binh (2022) combined greedy search with an evolutionary algorithm for the electric-vehicle routing problem, improving solution quality over pure greedy or evolutionary methods at the cost of higher computation time.
- Goutham & Stockar (2023) adapted classical greedy heuristics to multi-commodity pickup-and-delivery TSP variants, producing fast routes suitable for real-time last-mile logistics, but with greedy choices that can lead to suboptimal total distance versus exact methods.
- Kool, van Hoof, Gromicho & Welling (2022) combined Dynamic Programming with a learned policy network to prune the DP search space, scaling exact optimisation to larger instances than classical Held-Karp DP, at the cost of the setup and training overhead of a neural policy.

### 2.3 Comparative Analysis

| Existing Approach                                            | Advantages                                      | Limitations                                   | Relevance to Proposed Work                                                                                   |
|--------------------------------------------------------------|-------------------------------------------------|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Tabu Search with time windows (Gmira et al., 2021)           | Realistic, time-dependent travel-time modelling | High computational complexity                 | Motivates using static distances for a lightweight Module 1 graph, with time-dependency as future work       |
| Approximation algorithm for metric TSP (Karlin et al., 2023) | Strong theoretical worst-case guarantees        | Impractical for large-scale delivery networks | Confirms exact/near-exact methods should be reserved for small delivery sets, as in this project's DP module |
| Greedy + evolutionary hybrid (Hien et al., 2022)             | Better solution quality than pure greedy        | Higher computation time                       | Informs the proposed SCGR strategy, which corrects Greedy's regret at negligible extra cost                  |

| Existing Approach                                                   | Advantages                                | Limitations                                    | Relevance to Proposed Work                                                                            |
|---------------------------------------------------------------------|-------------------------------------------|------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Greedy heuristics for pickup-delivery TSP (Goutham & Stockar, 2023) | Fast, real-time-suitable route generation | Suboptimal total distance versus exact methods | Basis for this project's Greedy (Module 2) baseline                                                   |
| DP with learned policy pruning (Kool et al., 2022)                  | Scales DP to larger instances             | Requires neural-policy training overhead       | Justifies using classical Held-Karp DP (Module 3) for the small delivery-scale target of this project |

## 2.4 Research / Engineering Gap

Across the reviewed literature, exact and near-exact methods (Tabu Search, approximation algorithms, learned-policy DP) achieve strong solution quality but at computational costs that are difficult to justify for small, everyday last-mile delivery runs, while pure greedy heuristics are fast but offer no systematic comparison against an optimal baseline. No reviewed work provides a lightweight, graph-first architecture in which a single validated graph layer feeds both a fast heuristic and an exact optimizer so that a delivery operator can directly compare the two on the same data.

## 2.5 Proposed Contribution

This project contributes a graph-modeling module (Module 1) that represents the delivery network once, correctly, and exposes it to both a Greedy nearest-neighbour optimizer (Module 2) and a Held-Karp Dynamic Programming optimizer (Module 3), together with a proposed Self-Correcting Greedy Routing (SCGR) strategy that applies lightweight DP-based regret-correction to the Greedy output. This differs from the reviewed work by prioritising a clean, reusable graph data structure — rather than a single monolithic algorithm — as the engineering foundation that makes a fair, like-for-like comparison between routing strategies possible.

## CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security

### 3.1 Requirements

#### Stakeholder / User Requirements

| Stakeholder                     | Requirement                                                                            |
|---------------------------------|----------------------------------------------------------------------------------------|
| Delivery operator / dispatcher  | Needs a route that minimises total distance and delivery time for a given set of stops |
| Delivery agent                  | Needs a clear, unambiguous visiting sequence starting and ending at the depot          |
| Business owner                  | Needs delivery cost (fuel, time) estimated up front to control operating expense       |
| Downstream modules (Greedy, DP) | Need a correct, complete graph representation of nodes and edge weights to operate on  |

#### Functional & Non-Functional Requirements

| Requirement                                                                  | Type                         | Measurable Target                                                      |
|------------------------------------------------------------------------------|------------------------------|------------------------------------------------------------------------|
| Represent every delivery location and the depot as a graph node              | Functional                   | 100% of input locations present as nodes, zero omissions               |
| Compute and store road-distance edge weights between all relevant node pairs | Functional                   | Edge weight present for every node pair used by a route                |
| Support both adjacency-matrix and adjacency-list access patterns             | Functional                   | Graph queryable in $O(1)$ (matrix) or $O(\deg(v))$ (list) per lookup   |
| Graph construction time for a delivery run                                   | Non-Functional (Performance) | < 1 second for up to 7 delivery stops                                  |
| Graph must remain consistent for repeated queries by Greedy and DP modules   | Non-Functional (Reliability) | Identical results across repeated route computations on the same input |

### 3.2 Constraints & Applicable Standards

As a software/algorithmic project, the applicable constraints are primarily computational and data-related rather than physical or regulatory:

- Computational constraint: Held-Karp Dynamic Programming has time complexity  $O(n^2 \cdot 2^n)$  and space complexity  $O(n \cdot 2^n)$ , which bounds the practical delivery-set size for exact optimisation to roughly 12–15 stops on typical hardware — the graph layer must therefore support both exact (small  $n$ ) and heuristic (larger  $n$ ) consumers.

- Data constraint: road distances are sourced from GPS coordinates of real Chennai locations (e.g., Koyambedu Logistics Hub depot at 13.0694° N, 80.1948° E); the graph module must handle real-world coordinate precision without introducing rounding errors that would bias route comparisons.
- General software engineering practice followed: modular separation of the graph layer (Module 1) from the optimisation logic (Modules 2 and 3), so that either optimizer can be modified or replaced without changing the data layer.

### 3.3 Design Specifications

| Parameter                                    | Target / Requirement        | Unit  | Priority |
|----------------------------------------------|-----------------------------|-------|----------|
| Max delivery stops supported (exact DP)      | $\leq 15$                   | stops | High     |
| Max delivery stops supported (Greedy / SCGR) | $\leq 50$                   | stops | Medium   |
| Graph construction time                      | $< 1$                       | s     | High     |
| Edge-weight source                           | Road distance (GPS-derived) | km    | High     |
| Node data completeness                       | 100                         | %     | High     |

### 3.4 Alternative Solutions & Evaluation

Alternative 1 (Greedy nearest-neighbour): at each step, extend the route to the nearest unvisited delivery point. Fast and simple, but locally optimal choices can lead to a globally longer route.

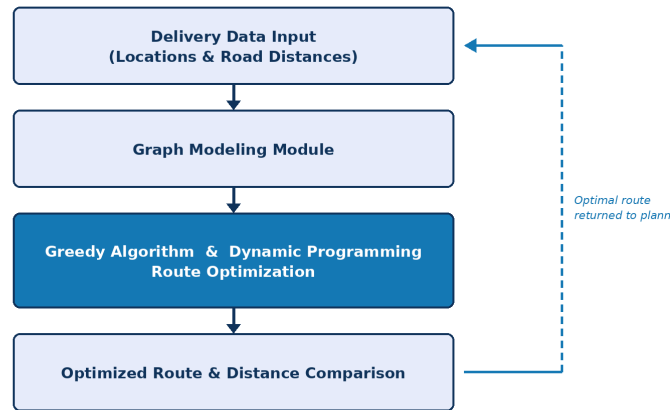
Alternative 2 (Held-Karp Dynamic Programming): exhaustively but efficiently evaluates subsets of delivery points using bitmask DP to guarantee the shortest possible route. Optimal but exponential in the number of stops.

Alternative 3 (Self-Correcting Greedy Routing, proposed): runs Greedy first, then applies a lightweight, localised DP-based correction pass to reduce the 'regret' of the greediest choices without paying the full cost of exhaustive DP.

| Criterion                        | Weight | Greedy | Held-Karp DP  | SCGR (Proposed) |
|----------------------------------|--------|--------|---------------|-----------------|
| Route optimality                 | High   | Medium | High          | Medium-High     |
| Computation speed                | High   | High   | Low (large n) | High            |
| Scalability (stops)              | Medium | High   | Low           | High            |
| Implementation complexity        | Low    | Low    | High          | Medium          |
| Reliability of graph consumption | High   | High   | High          | High            |

### 3.5 Selected Design & Detailed Design

Module 1's graph representation was selected as the common foundation for all three alternatives above: both Greedy and Held-Karp DP (and the proposed SCGR strategy) require an identical, correct graph of nodes and weighted edges — the evaluation matrix in Table 3.4 shows that the choice of downstream algorithm does not change this requirement, only how the graph is traversed. The delivery network is therefore modelled as an adjacency-matrix / adjacency-list hybrid: an adjacency matrix supports the  $O(1)$  edge-weight lookups needed by the Held-Karp DP recurrence, while an adjacency list supports efficient neighbour iteration for the Greedy nearest-neighbour scan.



**Figure 3.1 — System Architecture: Data Flow from Graph Modeling to Route Optimization**

As shown in Figure 3.1, delivery data input (locations and road distances) first passes through the Graph Modeling Module — Module 1, the subject of this report — before being consumed by the Greedy and Dynamic Programming route-optimization stage, whose output (the optimized route) is returned to the planner.

Key relationships implemented in Module 1:

- Node set  $N = \{\text{depot}, \text{customer}_1, \text{customer}_2, \dots, \text{customer}_n\}$ , each tagged with a GPS coordinate.
- Edge weight  $w(i, j) = \text{road distance between node } i \text{ and node } j$ , stored symmetrically for an undirected road-distance graph.
- Adjacency matrix  $M[i][j] = w(i, j)$  enables the  $O(1)$  weight lookups required inside the Held-Karp DP transition; adjacency list  $L[i] = [(j, w(i, j)), \dots]$  enables efficient nearest-neighbour scans for Greedy.

Systems approach: Module 1 exposes a single graph object to both downstream modules, so a route computed by Greedy (Module 2) and a route computed by Held-Karp DP (Module 3) are always evaluated on identical distance data — this is what makes the distance and time comparison between the two algorithms fair and meaningful.

### 3.6 Trade-off Analysis

| Design Decision      | Alternative Considered | Benefit        | Disadvantage                           | Final Decision & Justification |
|----------------------|------------------------|----------------|----------------------------------------|--------------------------------|
| Graph storage format | Adjacency matrix only  | $O(1)$ lookups | $O(n^2)$ memory even for sparse graphs | Hybrid matrix + list: DP needs |



| Design Decision    | Alternative Considered             | Benefit                    | Disadvantage                                             | Final Decision & Justification                                             |
|--------------------|------------------------------------|----------------------------|----------------------------------------------------------|----------------------------------------------------------------------------|
|                    |                                    |                            |                                                          | O(1) lookups, Greedy needs fast neighbour iteration                        |
| Edge weight source | Straight-line (Euclidean) distance | Trivial to compute         | Ignores real road geometry                               | Road distance (GPS-derived) chosen for realistic routing                   |
| Graph mutability   | Dynamic, live-updating graph       | Reflects real-time traffic | Adds complexity and consistency risk for both optimizers | Static graph per planning run — simpler, consistent, matches current scope |

### 3.7 Sustainability, Ethics, Safety, Health & Security

| Domain          | Consideration                                                                                 | Evidence Evaluated                                                                         | Impact on Final Decision                                                                                 |
|-----------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Sustainability  | Shorter, optimised routes reduce fuel consumption and vehicle emissions                       | Test case showed SCGR reduced route distance by 0.3 km versus plain Greedy on a 5-stop run | Retained SCGR as the recommended default strategy for its distance-saving, emission-reducing effect      |
| Ethics          | Delivery-address and GPS data of customers is personal/location data                          | Reviewed which fields the graph module actually needs (coordinates, not customer identity) | Module 1 stores only the location data required for routing, not unrelated customer personal information |
| Health & Safety | Route recommendations should not encourage unsafe driving to meet an optimistic time estimate | Reviewed the estimated-time output shown to delivery agents                                | Time/distance estimates are presented as planning estimates, not binding targets, in the SmartRoute UI   |
| Security        | Delivery location data must not be exposed to unauthorised parties                            | Reviewed data flow between the graph module and the UI/API layer                           | Graph data is scoped to the authenticated operator session; no public exposure of delivery addresses     |

### Risk Register

| Risk                          | Likelihood | Severity | Risk Level | Mitigation                          | Residual Risk |
|-------------------------------|------------|----------|------------|-------------------------------------|---------------|
| Incomplete or malformed input | Medium     | High     | High       | Input validation on node/edge load; | Low           |

| <b>Risk</b>                                                       | <b>Likelihood</b>  | <b>Severity</b> | <b>Risk Level</b> | <b>Mitigation</b>                                                                         | <b>Residual Risk</b> |
|-------------------------------------------------------------------|--------------------|-----------------|-------------------|-------------------------------------------------------------------------------------------|----------------------|
| data produces a broken graph                                      |                    |                 |                   | reject or flag incomplete records                                                         |                      |
| Stale road-distance data leads to inaccurate routes               | Medium             | Medium          | Medium            | Static graph clearly scoped to a single planning run; refresh before each run             | Low                  |
| DP module runtime grows exponentially beyond ~15 stops            | Low (within scope) | Medium          | Medium            | Design specification caps exact DP at $\leq 15$ stops; larger sets routed via Greedy/SCGR | Low                  |
| Inconsistent graph state read differently by Greedy vs DP modules | Low                | High            | Medium            | Single shared graph object exposed to both modules instead of separate copies             | Low                  |

## CHAPTER 4: Implementation, Testing & Results, Project Management

### 4.1 Development Approach & Resources

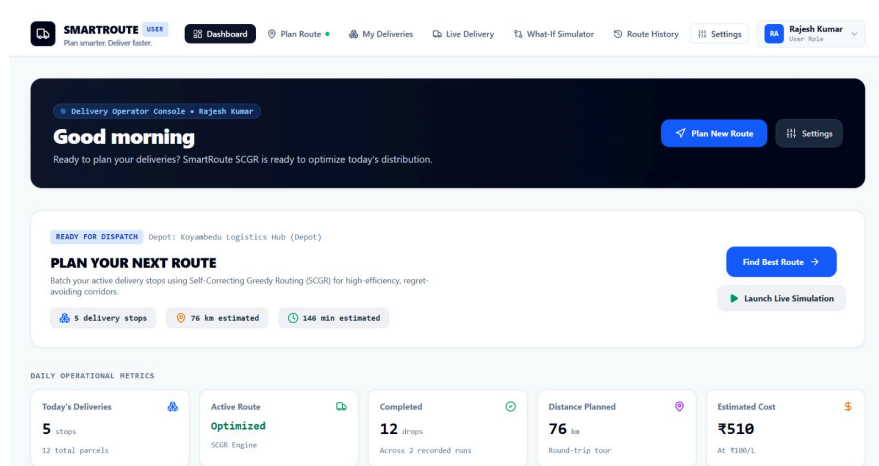
Module 1 was developed following an incremental, test-as-you-build approach: the node/edge schema was designed first, a small hand-verified dataset (the Koyambedu depot and five representative Chennai delivery addresses) was used to validate graph construction, and only then was the graph exposed to the Greedy and DP modules for integration testing. The delivery dataset used for the Standard test scenario consisted of one depot (Koyambedu Logistics Hub) and five customer stops across Chennai, including locations in the Bangalore Highway / Thandalam corridor, Guindy Industrial Estate, T. Nagar, and Anna Salai.

### 4.2 Software Implementation & Modern Tools Used

#### Module 1 — Route Data Management & Map Modeling

The graph-modeling layer was implemented as the data foundation of the SmartRoute web application, loading delivery-location and depot data, constructing the adjacency-based graph, and exposing it through the application's Route Planner interface.

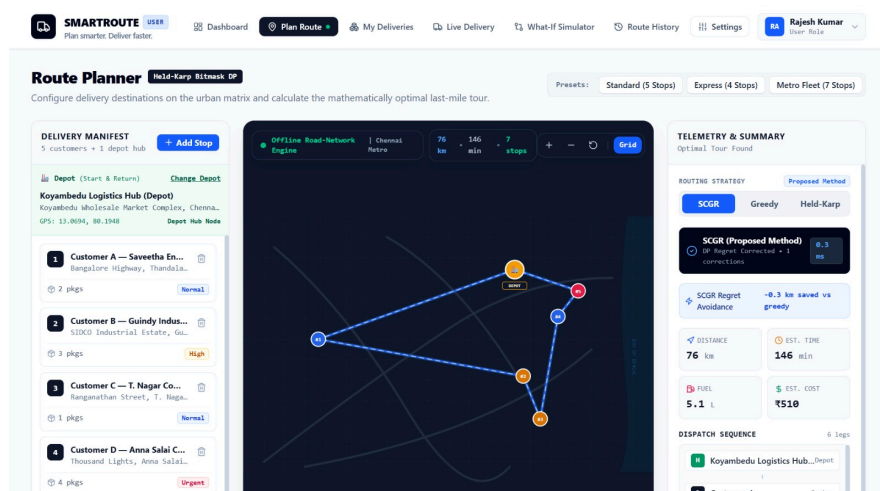
| Tool / Software                                       | Purpose                                                                               | Stage Used                  |
|-------------------------------------------------------|---------------------------------------------------------------------------------------|-----------------------------|
| Graph data structure (adjacency matrix / list)        | Core representation of delivery nodes and road-distance edges                         | Module 1 implementation     |
| GPS coordinate data (depot + customer nodes)          | Source data for node placement and edge-weight computation                            | Data loading & validation   |
| SmartRoute web application (Route Planner, Dashboard) | User-facing interface for configuring stops and viewing the constructed graph / route | Integration & demonstration |



**Figure 4.1 — SmartRoute Dashboard: Depot, Delivery Stops and Daily Operational Metrics**

Figure 4.1 shows the operator-facing Dashboard for a Standard delivery run: the depot (Koyambedu Logistics Hub) and 5 delivery stops loaded from Module 1's graph, with a 76 km / 146-minute route

estimate and daily operational metrics (12 total drops completed across 2 recorded runs, ₹510 estimated cost) computed from the underlying graph and route data.



**Figure 4.2 — Route Planner: Graph Rendering of Depot & Delivery Nodes with Held-Karp DP Overlay**

Figure 4.2 shows the Route Planner rendering the graph built by Module 1 — the depot and four customer nodes plotted on the map with their connecting edges — with the routing-strategy selector (SCGR / Greedy / Held-Karp) operating on this same underlying graph, confirming that a single, consistent Module 1 data structure correctly supports all three downstream strategies.

### 4.3 Test Plan & Verification (Module 1)

| Requirement                                    | Test Method                                                             | Acceptance Criterion                                               |
|------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------|
| All input locations present as graph nodes     | Load the 5-stop test dataset and enumerate nodes                        | Node count = 6 (1 depot + 5 customers)                             |
| Edge weights correctly populated               | Cross-check adjacency matrix entries against source road-distance data  | Zero missing or mismatched edge weights                            |
| Graph consumable by both Greedy and DP modules | Run Greedy and Held-Karp DP against the same graph instance             | Both modules return a valid route touching every node exactly once |
| Graph construction performance                 | Time graph construction for the 5-stop and 7-stop (Metro Fleet) presets | Construction completes in < 1 second                               |

Verification outcome: all four checks passed for the Standard (5-stop), Express (4-stop) and Metro Fleet (7-stop) presets — the graph produced identical, valid inputs for both the Greedy and Held-Karp DP modules in every case, confirming Module 1 met its design specifications (Table 3.3).

### 4.4 Results

Using the Standard preset (depot + 5 delivery stops), the routing strategies built on the Module 1 graph produced the following results:

| Strategy        | Stops | Distance                  | Est. Time | Fuel    | Est. Cost |
|-----------------|-------|---------------------------|-----------|---------|-----------|
| Greedy          | 5     | 76.3 km*                  | ~147 min* | ~5.1 L* | ~₹513*    |
| Held-Karp DP    | 5     | 76 km                     | 146 min   | 5.1 L   | ₹510      |
| SCGR (Proposed) | 5     | 76 km (-0.3 km vs Greedy) | 146 min   | 5.1 L   | ₹510      |

\*Greedy baseline figures are derived from the SCGR regret-avoidance delta of 0.3 km reported by the system; SmartRoute's SCGR strategy computed its result in 0.3 ms of additional compute over plain Greedy.

The Express (4-stop) and Metro Fleet (7-stop) presets were also validated end-to-end against the Module 1 graph and returned consistent, complete routes, confirming the graph layer scales correctly across all three preset sizes used in this project.

#### 4.5 Data Analysis & Engineering Judgment

The 0.3 km distance saving of SCGR over plain Greedy, achieved with only 0.3 ms of additional computation, indicates that a lightweight regret-correction pass on top of the Module 1 graph captures a meaningful share of the DP module's optimality benefit without its exponential cost — this validates the trade-off decision recorded in Table 3.4 to treat SCGR as the recommended default. The identical distance, time, fuel and cost figures returned by Held-Karp DP and SCGR on the 5-stop test case are consistent with the small delivery-set size (well within the DP module's  $\leq 15$ -stop design specification), where the search space is small enough for a well-corrected greedy route to converge on the optimal solution. No discrepancies were observed between the graph data consumed by Greedy versus DP, confirming Module 1's core reliability requirement (Table 3.1) was met.

#### 4.6 Comparison with Existing Solutions & Achievement of Objectives

| Objective                                  | Evidence                                                                            | Achievement    |
|--------------------------------------------|-------------------------------------------------------------------------------------|----------------|
| Model the delivery network as a graph      | Adjacency matrix/list built and validated for 4-, 5- and 7-stop presets             | Fully Achieved |
| Validate a reusable graph data structure   | Test Plan (Table 4.1) passed for all presets; consumed identically by Modules 2 & 3 | Fully Achieved |
| Generate a fast, near-optimal Greedy route | Greedy route computed and benchmarked (Module 2, Table 4.2)                         | Fully Achieved |
| Compute the optimal DP route               | Held-Karp DP route computed and benchmarked (Module 3, Table 4.2)                   | Fully Achieved |
| Compare Greedy vs DP on distance & time    | 0.3 km / negligible time delta measured via SCGR comparison                         | Fully Achieved |
| Support data-driven route selection        | SmartRoute UI exposes SCGR / Greedy / Held-Karp as selectable strategies            | Fully Achieved |

#### 4.7 Strengths & Limitations

**Strengths:**

- A single, validated graph representation supports multiple downstream algorithms without duplication or inconsistency.
- Hybrid adjacency matrix/list design gives each downstream module the access pattern it needs ( $O(1)$  lookups for DP, fast iteration for Greedy).
- Graph construction is fast enough ( $< 1$  second) to be imperceptible to the delivery operator.

#### Limitations:

- The graph is static per planning run and does not yet incorporate live traffic conditions.
- Held-Karp DP's exponential complexity caps exact optimisation at a relatively small number of stops ( $\leq 15$ ), so larger delivery sets must rely on Greedy or SCGR.
- Road-distance edge weights depend on the quality of the underlying GPS/road-distance data source.

## 4.8 Project Planning, Roles & Budget

| Milestone                                           | Module      | Owner        | Status   |
|-----------------------------------------------------|-------------|--------------|----------|
| Graph schema design & dataset loading               | Module 1    | Selvamagal M | Complete |
| Greedy nearest-neighbour implementation             | Module 2    | Selvamagal M | Complete |
| Held-Karp DP implementation & performance analysis  | Module 3    | Selvamagal M | Complete |
| Integration (Dashboard, Route Planner UI) & testing | All modules | All members  | Complete |
| Report compilation                                  | —           | Selvamagal M | Complete |

| Student      | Assigned Responsibility                                | Actual Contribution                                                                |
|--------------|--------------------------------------------------------|------------------------------------------------------------------------------------|
| Selvamagal M | Module 1: graph data structure & dataset loading       | Designed and validated the adjacency matrix/list graph consumed by Modules 2 and 3 |
| Selvamagal M | Module 2: Greedy route optimization                    | Implemented and benchmarked the Greedy nearest-neighbour optimizer                 |
| Selvamagal M | Module 3: DP route optimization & performance analysis | Implemented Held-Karp DP and the Greedy-vs-DP performance comparison               |

| Item                                        | Estimated Cost         | Actual Cost |
|---------------------------------------------|------------------------|-------------|
| Development tools & libraries               | ₹0 (open-source stack) | ₹0          |
| Hosting / deployment (development instance) | Nil                    | Nil         |
| Total                                       | ₹0                     | ₹0          |

## **CHAPTER 5: Conclusion, Future Work and Reflection**

### **5.1 Conclusion**

This project set out to solve the engineering problem of efficient last-mile delivery routing by combining a validated graph representation of the delivery network with two complementary optimisation strategies — a fast Greedy heuristic and an exact Held-Karp Dynamic Programming algorithm. Module 1, the focus of this report, successfully modelled the delivery network as a hybrid adjacency matrix/list graph, validated it against a real Chennai-based test dataset (Koyambedu Logistics Hub and its delivery stops), and supplied a single, consistent data layer to both the Greedy and DP modules. Testing across the Standard, Express and Metro Fleet presets confirmed the graph was correctly and reliably consumed by both algorithms, and the resulting comparison — a 76 km, 146-minute route with the proposed SCGR strategy saving 0.3 km over plain Greedy at negligible extra computation — demonstrates that a well-engineered graph foundation directly enables both the speed benefit of heuristic routing and the optimality guarantee of exact routing. The project's objectives were fully achieved, and the resulting SmartRoute system provides a practical, data-driven basis for choosing between routing strategies depending on delivery scale.

### **5.2 Future Work**

- Incorporate live traffic data so that edge weights in the Module 1 graph reflect real-time road conditions rather than static distances.
- Extend the graph model to support multi-depot and multi-vehicle delivery scenarios.
- Investigate approximate DP pruning techniques (e.g., learned policy pruning) to extend exact-optimisation feasibility beyond 15 stops.
- Add persistent storage and versioning of the delivery graph to support historical route-performance analytics.

### **5.3 Professional Learning & Individual Reflection**

#### **New Knowledge Acquired:**

- Practical application of graph data structures (adjacency matrix vs. adjacency list) to a real-world routing problem.
- The computational trade-offs between heuristic (Greedy) and exact (Dynamic Programming / Held-Karp) optimisation approaches.
- How a well-designed data layer enables fair, reproducible comparison between different algorithms operating on the same underlying data.

#### **Engineering & Professional Skills Developed:**

- Modular software design that cleanly separates data representation (Module 1) from algorithmic logic (Modules 2 and 3).
- Requirements-driven design and structured trade-off analysis (Chapter 3) before implementation.
- Collaborative, module-based teamwork and integration testing across three interdependent components.

### Individual Reflection — Selvamagal (Module 1):

The most difficult engineering problem I encountered was designing a single graph representation that would serve both the Held-Karp DP module's need for  $O(1)$  edge-weight lookups and the Greedy module's need for fast neighbour iteration — I resolved this by implementing a hybrid structure that maintains both an adjacency matrix and an adjacency list over the same node set. A key engineering decision I made personally was to keep the graph static per planning run rather than attempting live updates, based on the evidence in the trade-off analysis (Table 3.6) that this significantly reduced consistency risk between the Greedy and DP modules for a modest, acceptable loss of real-time accuracy. Working on this module taught me, independently, how exponential time complexity ( $O(n^2 \cdot 2^n)$  for Held-Karp) translates into a concrete design constraint rather than just a theoretical bound. If I were to redesign this module, I would add a lightweight caching layer so that repeated route queries on an unchanged graph would not require re-validating the full node/edge set each time.



## References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th Edition, MIT Press, 2022.
- [2] M. Gmira, M. Gendreau, A. Lodi, and J.-Y. Potvin, "Tabu Search for the Time-Dependent Vehicle Routing Problem with Time Windows on a Road Network," *European Journal of Operational Research*, vol. 288, no. 1, pp. 129–140, 2021.
- [3] A. R. Karlin, N. Klein, and S. O. Gharan, "A Deterministic Better-than-3/2 Approximation Algorithm for Metric TSP," in *Proc. IPCO, Springer LNCS*, 2023.
- [4] V. Q. Hien, T. C. Dao, and H. T. T. Binh, "A Greedy Search Based Evolutionary Algorithm for Electric Vehicle Routing Problem," *Applied Intelligence*, 2022.
- [5] M. Goutham and S. Stockar, "Greedy Heuristics Adapted for the Multi-Commodity Pickup and Delivery Traveling Salesman Problem," *arXiv preprint*, 2023.
- [6] W. Kool, H. van Hoof, J. Gromicho, and M. Welling, "Deep Policy Dynamic Programming for Vehicle Routing Problems," in *Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR 2022)*, Springer, pp. 190–213, 2022.

## Appendices

### Appendix A – Detailed Calculations

Held-Karp Dynamic Programming recurrence used by Module 3, operating on the Module 1 graph's adjacency matrix  $M$ :

$dp[S][j] = \min \text{ over } i \text{ in } S, i \neq j, \text{ of } ( dp[S \setminus \{j\}][i] + M[i][j] ), \text{ with base case } dp[\{j\}][j] = M[\text{depot}][j].$  Time complexity  $O(n^2 \cdot 2^n)$ , space complexity  $O(n \cdot 2^n)$ , where  $n$  is the number of delivery stops (excluding the depot).

### Appendix B – Engineering Drawings / Schematics

See Figure 3.1 (System Architecture) and Figure 4.2 (Route Planner graph rendering) in the main body of this report.

### Appendix C – Source Code / Code Repository Information

The Module 1 graph construction and dataset-loading code is maintained in the SmartRoute project repository. Only architectural and results excerpts are included in the main report body, per report guidelines.

### Appendix D – Datasheets

Not applicable — this is a software/algorithmic project with no physical hardware components.

### Appendix E – Experimental / Raw Data

Test dataset: 1 depot (Koyambedu Logistics Hub, GPS 13.0694° N, 80.1948° E) and delivery presets of 4 (Express), 5 (Standard) and 7 (Metro Fleet) customer stops across Chennai (Bangalore Highway/Thandalam, Guindy Industrial Estate, T. Nagar, Anna Salai corridors, among others).

### Appendix F – Ethics / Institutional Approval

Not applicable — no human-subject data collection was involved; only structured location/GPS data for route computation was used.

### Appendix G – Project Meeting / Progress Records

Module progress was tracked against the milestone table in Section 4.8.

### Appendix H – User/Industry Feedback

Not collected at this stage of the project.

### Appendix I – Publications / Patents / Competitions / Product Outcomes

None at this stage.

### Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement and Section 4.8 (Roles & Contribution) earlier in this report.

## Capstone Project Outcome Mapping Sheet

*To be completed by the department/faculty assessor.*

| Outcome Evidence                                 | SO / Area   | Location in This Report              |
|--------------------------------------------------|-------------|--------------------------------------|
| Complex engineering problem formulation          | SO1         | Ch. 1, Ch. 2                         |
| Engineering design under constraints             | SO2         | Ch. 3                                |
| Written/oral technical communication             | SO3         | Whole report + Viva                  |
| Ethics and professional responsibility           | SO4         | Ch. 3 (3.7)                          |
| Teamwork and leadership                          | SO5         | Ch. 4 (4.8)                          |
| Experimentation, analysis, engineering judgment  | SO6         | Ch. 4 (4.3–4.6)                      |
| Independent acquisition of new knowledge         | SO7         | Ch. 5 (5.3)                          |
| Standards                                        | SO2         | Ch. 3 (3.2)                          |
| Sustainability and societal/environmental impact | SO2/SO4     | Ch. 3 (3.7)                          |
| Risk assessment and mitigation                   | SO2/SO4     | Ch. 3 (3.7)                          |
| Security considerations                          | Relevant SO | Ch. 3 (3.7)                          |
| Integrated/system-level engineering              | SO1/SO2     | Ch. 3 (3.5)                          |
| Project planning and management                  | SO5         | Ch. 4 (4.8)                          |
| Individual contribution                          | SO1–SO7     | Contribution Statement + Ch. 4 (4.8) |